# ANALYSING E-LEARNING PLATFORM PURCHASES USING MYSQL

## PROJECT OVERVIEW:

The project *"**Analysing E-Learning Platform Purchases using MySQL**"* aims to design and query a relational database to study learner purchases across countries. It integrates learner, course, and purchase data to uncover sales trends, learner behaviour, and category performance. SQL techniques such as joins, aggregations, subqueries, CTEs, and views are applied to generate actionable insights for management.

---

## PROJECT OBJECTIVES:

1. Build a normalized database with learners, courses, and purchases, ensuring proper keys and relationships.
2. Explore data using joins and aggregations to analyse spending, popular courses, and category revenues.
3. Apply advanced SQL (subqueries, CTEs, CASE, NULL handling) to classify learners and detect patterns.
4. Summarize findings in a report with clear insights and recommendations for improving course sales and engagement.

---

## DATABASE SCHEMA:

Database Name: *eLearning _db* This database stores all information related to learners, courses, and their purchases.

### 1. Learners Table:

Columns:

- ***Learner ID*:** Unique identifier for each learner (Primary Key).
- ***Full Name*:** Name of the learner.
- ***Country*:** Country of residence.

### 2. Courses Table:

Columns:

- ***Course ID*:** Unique identifier for each course (Primary Key).
- ***Course Name*:** Title of the course.
- ***Category*:** The category to which the course belongs (e.g., Beginner, Advanced, Technical).
- ***Unit Price*:** Price of the course.

### 3. Purchases Table

Columns:

- ***Purchase ID*:** Unique identifier for each purchase (Primary Key).
- ***Learner ID***: References the learner who made the purchase (Foreign Key linked to Learners table).
- ***Course ID***: References the course purchased (Foreign Key linked to Courses table).
- ***Quantity*:** Number of units purchased.
- ***Purchase Date*:** Date when the purchase was made.

---

# 1. DATABASE SETUP & DATA ENTRY:

## DATA BASE CREATION:

The database was created and named ***elearning*_db** to store and analyse learner purchases, courses, and related insights.

```
-- Step 1: Create Database
CREATE DATABASE elearning_db;
USE elearning_db;
```

## TABLE CREATION:

The database schema created for this analysis includes three tables: **Learners, Courses and Purchases**

```
-- Step 2: Create Learners Table
CREATE TABLE learners (
    learner_id INT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    country VARCHAR(50) NOT NULL
);

-- Step 3: Create Courses Table
CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL UNIQUE,
    category VARCHAR(50) NOT NULL,
    unit_price DECIMAL(8,2) NOT NULL CHECK (unit_price > 0)
);

-- Step 4: Create Purchases Table
CREATE TABLE purchases (
    purchase_id INT PRIMARY KEY,
    learner_id INT,
    course_id INT,
    quantity INT CHECK (quantity > 0),
    purchase_date DATE NOT NULL,
    FOREIGN KEY (learner_id)
    REFERENCES learners(learner_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (course_id)
    REFERENCES courses(course_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

**INSERT SAMPLE DATA:**

To demonstrate the database functionality, sample records were added across all three tables

```
37    -- Step 5: Insert Sample Learners
38 •  INSERT INTO learners (learner_id, full_name, country) VALUES
39    (1, 'Arun Kumar', 'India'),
40    (2, 'Meena R', 'USA'),
41    (3, 'Kiran S', 'UK'),
42    (4, 'Divya P', 'Canada'),
43    (5, 'Rahul T', 'Australia');
44
45    -- Step 6: Insert Sample Courses
46 •  INSERT INTO courses (course_id, course_name, category, unit_price) VALUES
47    (101, 'Python Basics', 'Programming', 50000.00),
48    (102, 'Data Science Fundamentals', 'Data Science', 75000.00),
49    (103, 'Digital Marketing', 'Marketing', 40000.00),
50    (104, 'Machine Learning', 'AI', 100000.00),
51    (105, 'Web Development', 'Programming', 60000.00);
52
53
54    -- Step 7: Insert Sample Purchases
55 •  INSERT INTO purchases (purchase_id, learner_id, course_id, quantity, purchase_date) VALUES
56    (1001, 1, 101, 1, '2026-06-01'),
57    (1002, 1, 104, 1, '2026-06-02'),
58    (1003, 2, 102, 2, '2026-06-03'),
59    (1004, 3, 103, 1, '2026-06-04'),
60    (1005, 4, 105, 3, '2026-06-05'),
61    (1006, 5, 101, 2, '2026-06-06'),
62    (1007, 2, 104, 1, '2026-06-07'),
63    (1008, 3, 105, 1, '2026-06-08');
64
```

## 2. DATA EXPLORATION USING JOINS

Displaying Learner name, Course name, Category, Quantity, Total amount, Purchase date using **inner join**

```
-- Display: Learner name, Course name, Category, Quantity, Total amount, Purchase date
select
l.full_name as leaners_name,
c.course_name as coursename,
c.category as cource_category,
p.quantity as purchase_quantity,
 round(p.quantity*c.unit_price,2) as total_amount,
 p.purchase_date as purchasedate
 from learners as l
 inner join
 purchases as p
 on p.learner_id=l.learner_id
 inner join
 courses as c
 on c.course_id=p.course_id
 order by total_amount desc;
```

esult Grid | Filter Rows: | Export: | Wrap Cell Content:

| leaners_name | coursename | cource_category | purchase_quantity | total_amount | purchasedate |
|---|---|---|---|---|---|
| Divya P | Web Development | Programming | 3 | 18000.00 | 2026-06-05 |
| Meena R | Data Science Fundamentals | Data Science | 2 | 15000.00 | 2026-06-03 |
| Arun Kumar | Machine Learning | AI | 1 | 10000.00 | 2026-06-02 |
| Meena R | Machine Learning | AI | 1 | 10000.00 | 2026-06-07 |
| Rahul T | Python Basics | Programming | 2 | 10000.00 | 2026-06-06 |
| Kiran S | Web Development | Programming | 1 | 6000.00 | 2026-06-08 |
| Arun Kumar | Python Basics | Programming | 1 | 5000.00 | 2026-06-01 |
| Kiran S | Digital Marketing | Marketing | 1 | 4000.00 | 2026-06-04 |

Displaying Learner name, Course name, Category, Quantity, Total amount, Purchase date using **left join**

```
 -- left join
select
l.full_name as leaners_name,
c.course_name as coursename,
c.category as cource_category,
p.quantity as purchase_quantity,
 round(p.quantity*c.unit_price,2) as total_amount,
 p.purchase_date as purchasedate
 from purchases as p
left join learners as l
 ON p.learner_id = l.learner_id
left join courses as c
on p.course_id = c.course_id
order by Total_Amount DESC;
```

ult Grid | Filter Rows: | Export: | Wrap Cell Content:

| leaners_name | coursename | cource_category | purchase_quantity | total_amount | purchasedate |
|---|---|---|---|---|---|
| Divya P | Web Development | Programming | 3 | 18000.00 | 2026-06-05 |
| Meena R | Data Science Fundamentals | Data Science | 2 | 15000.00 | 2026-06-03 |
| Arun Kumar | Machine Learning | AI | 1 | 10000.00 | 2026-06-02 |
| Rahul T | Python Basics | Programming | 2 | 10000.00 | 2026-06-06 |
| Meena R | Machine Learning | AI | 1 | 10000.00 | 2026-06-07 |

ult 1 ×

Displaying Learner name, Course name, Category, Quantity, Total amount, Purchase date using **right join**

```
-- right join
select
l.full_name as leaners_name,
c.course_name as coursename,
c.category as cource_category,
p.quantity as purchase_quantity,
 round(p.quantity*c.unit_price,2) as total_amount,
 p.purchase_date as purchasedate
 from  purchases as p
right join courses as c
on p.course_id = c.course_id
right join learners as l
 on  p.learner_id = l.learner_id
order by  total_Amount DESC;

--  Core Analytical Queries
```

sult Grid | Filter Rows: | Export: | Wrap Cell Content:

| leaners_name | coursename | cource_category | purchase_quantity | total_amount | purchasedate |
|---|---|---|---|---|---|
| Divya P | Web Development | Programming | 3 | 18000.00 | 2026-06-05 |
| Meena R | Data Science Fundamentals | Data Science | 2 | 15000.00 | 2026-06-03 |
| Arun Kumar | Machine Learning | AI | 1 | 10000.00 | 2026-06-02 |
| Meena R | Machine Learning | AI | 1 | 10000.00 | 2026-06-07 |
| Rahul T | Python Basics | Programming | 2 | 10000.00 | 2026-06-06 |

ult 2 ×

---

## 3. CORE ANALYTICAL QUERIES:

Q1. Displaying each learner's total spending with their country.

```
--  Core Analytical Queries
-- Q1. Display each learner's total spending with their country.
select
    l.full_name as learnername,
    l.country as country,
    round(sum(p.quantity * c.unit_price), 2) as totalspending
from purchases p
inner join learners l on p.learner_id = l.learner_id
inner join courses c on p.course_id = c.course_id
group by l.full_name, l.country
order by totalspending desc;
```

esult Grid | Filter Rows: | Export: | Wrap Cell Content:

| learnername | country | totalspending |
|---|---|---|
| Meena R | USA | 25000.00 |
| Divya P | Canada | 18000.00 |
| Arun Kumar | India | 15000.00 |
| Kiran S | UK | 10000.00 |
| Rahul T | Australia | 10000.00 |

esult 3 ×

Q2. Finding the top 3 most purchased courses by quantity.

```
124     -- find the top 3 most purchased courses by quantity.
125
126 •   select
127         c.course_name as coursename,
128         sum(p.quantity) as totalquantity
129     from purchases p
130     inner join courses c on p.course_id = c.course_id
131     group by c.course_name
132     order by totalquantity desc
133     limit 3;
134
135
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows

| coursename | totalquantity |
|---|---|
| Web Development | 4 |
| Python Basics | 3 |
| Data Science Fundamentals | 2 |

Q3. Showing each category's:

- Total revenue
- Number of unique learners

```
136     --  Show each category's: Total revenue , Number of unique learners
137 •   select
138     c.category,
139     sum(p.quantity*c.unit_price) as total_revenue,
140     count(distinct p.learner_id )as unique_learners
141     from courses as c
142     join
143     purchases as p
144     on p.course_id=c.course_id
145     group by c.category
146     order by  unique_learners;
147
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| category | total_revenue | unique_learners |
|---|---|---|
| Data Science | 15000.00 | 1 |
| Marketing | 4000.00 | 1 |
| AI | 20000.00 | 2 |
| Programming | 39000.00 | 4 |

Q4. displaying learners who purchased from more than one category.

```
148     -- List learners who purchased from more than one category.
149 •   select
150         l.full_name as learnername,
151         count(c.category) as categorycount
152     from purchases p
153     inner join learners l
154     on p.learner_id = l.learner_id
155     inner join courses c
156     on p.course_id = c.course_id
157     group by l.full_name
158     having count(distinct c.category) > 1
159     order by categorycount desc;
160
161     -- identify courses never purchased.
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| learnername | categorycount |
|---|---|
| Arun Kumar | 2 |
| Kiran S | 2 |
| Meena R | 2 |

Q5. Identifying courses never purchased.

```
161     -- identify courses never purchased.
162 •   select
163         c.course_id,
164         c.course_name,
165         c.category,
166         c.unit_price
167     from courses c
168     left join purchases p on c.course_id = p.course_id
169     where p.course_id is null
170     order by c.course_name;
171
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| course_id | course_name | category | unit_price |
|---|---|---|---|

## 4. SUBQUERIES & CORRELATED SUBQUERIES:

Q6. Finding learners whose total spending is above the average learner spending.

```
73  -- q6. find learners whose total spending is above the average learner spending.
74  select
75      l.full_name as learnername,
76      sum(p.quantity * c.unit_price) as totalspending
77  from purchases as p
78  inner join learners as l
79  on p.learner_id = l.learner_id
80  inner join courses as c
81  on p.course_id = c.course_id
82  group by l.full_name
83  having sum(p.quantity * c.unit_price)
84   > (
85      select avg(learnertotal)
86      from (
87          select sum(p2.quantity * c2.unit_price) as learnertotal
88          from purchases as p2
89          inner join courses as c2 on p2.course_id = c2.course_id
90          group by p2.learner_id
91      ) as subquery
92  );
```

esult Grid | Filter Rows: | Export: | Wrap Cell Content:

| learnername | totalspending |
|---|---|
| Meena R | 25000.00 |
| Divya P | 18000.00 |

Q7. Displaying courses whose price is higher than any course in the ‘Beginner’ category.

```
94  -- 7. display courses whose price is higher than any course in the 'marketing' category.
95  select
96      c.course_id,
97      c.course_name,
98      c.category,
99      c.unit_price
00  from courses c
01  where c.unit_price > (
02      select max(unit_price)
03      from courses
04      where category = 'marketing'
05  )
06  order by c.unit_price desc;
```

esult Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

| course_id | course_name | category | unit_price |
|---|---|---|---|
| 104 | Machine Learning | AI | 10000.00 |
| 102 | Data Science Fundamentals | Data Science | 7500.00 |
| 105 | Web Development | Programming | 6000.00 |
| 101 | Python Basics | Programming | 5000.00 |
| NULL | NULL | NULL | NULL |

urses 10 ×

Q8. Finding learners who spent more than the average spending in their country.

```sql
-- 8. find learners who spent more than the average spending in their country.
select
    l.full_name ,
    l.country ,
    sum(p.quantity * c.unit_price) as totalspending
from purchases p
inner join learners l on p.learner_id = l.learner_id
inner join courses c on p.course_id = c.course_id
group by l.full_name, l.country
having sum(p.quantity * c.unit_price) > (
    select avg(countrytotal)
    from (
        select
            l2.country,
            sum(p2.quantity * c2.unit_price) as countrytotal
        from purchases p2
        inner join learners l2 on p2.learner_id = l2.learner_id
        inner join courses c2 on p2.course_id = c2.course_id
        where l2.country = l.country
        group by l2.learner_id, l2.country
    ) as subquery
);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| full_name | country | totalspending |
|---|---|---|

## 5. CTE, CASE, VIEW, AND NULL HANDLING

Q9. Using a CTE to calculate total spending per learner, then: Displaying learners with spending above 10,000.

```sql
-- 9. use a cte to calculate total spending per learner, then display learners with spending above 10,000.
with learner_spending as (
    select
        l.learner_id,
        l.full_name,
        sum(p.quantity * c.unit_price) as totalspending
    from purchases p
    inner join learners l on p.learner_id = l.learner_id
    inner join courses c on p.course_id = c.course_id
    group by l.learner_id, l.full_name
)
select
    full_name,
    totalspending
from learner_spending
where totalspending > 10000
order by totalspending desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| full_name | totalspending |
|---|---|
| Meena R | 25000.00 |
| Divya P | 18000.00 |
| Arun Kumar | 15000.00 |

Q10. CASE Expression Classify learners based on spending:

```
-- q10. case expression classify learners based on spending.
select
    l.full_name,
    sum(p.quantity * c.unit_price) as totalspending,
    case
        when sum(p.quantity * c.unit_price) > 15000 then 'high value'
        when sum(p.quantity * c.unit_price) between 8000 and 15000 then 'medium value'
        else 'low value'
    end as learner_category
from purchases p
inner join learners l on p.learner_id = l.learner_id
inner join courses c on p.course_id = c.course_id
group by l.full_name
order by totalspending desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| full_name | totalspending | learner_category |
|---|---|---|
| Meena R | 25000.00 | high value |
| Divya P | 18000.00 | high value |
| Arun Kumar | 15000.00 | medium value |
| Kiran S | 10000.00 | medium value |
| Rahul T | 10000.00 | medium value |

Q11. NULL Handling ● Display all courses and replace NULL purchase counts with 0 using: COALESCE()

```
-- q11. null handling display all courses and replace null purchase counts with 0.
select
    c.course_id,
    c.course_name,
    c.category,
    coalesce(count(p.purchase_id), 0) as purchase_count
from courses c
left join purchases p on c.course_id = p.course_id
group by c.course_id, c.course_name, c.category
order by c.course_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| course_id | course_name | category | purchase_count |
|---|---|---|---|
| 102 | Data Science Fundamentals | Data Science | 1 |
| 103 | Digital Marketing | Marketing | 1 |
| 104 | Machine Learning | AI | 2 |
| 101 | Python Basics | Programming | 2 |
| 105 | Web Development | Programming | 2 |

Q12. View Deliverables

● Creating a view: category_performance_view

● Showing: ● Category ● Total revenue ● Number of purchases

```
75    -- q12. view create a view: category_performance_view.
76 •  create view category_performance_view as
77    select
78        c.category,
79        round(sum(p.quantity * c.unit_price), 2) as total_revenue,
80        count(p.purchase_id) as number_of_purchases,
81        round(avg(p.quantity * c.unit_price), 2) as avg_revenue_per_purchase
82    from purchases p
83    inner join courses c on p.course_id = c.course_id
84    group by c.category;
85
86 •  select * from category_performance_view;
```

esult Grid | Filter Rows: | Export: | Wrap Cell Content:

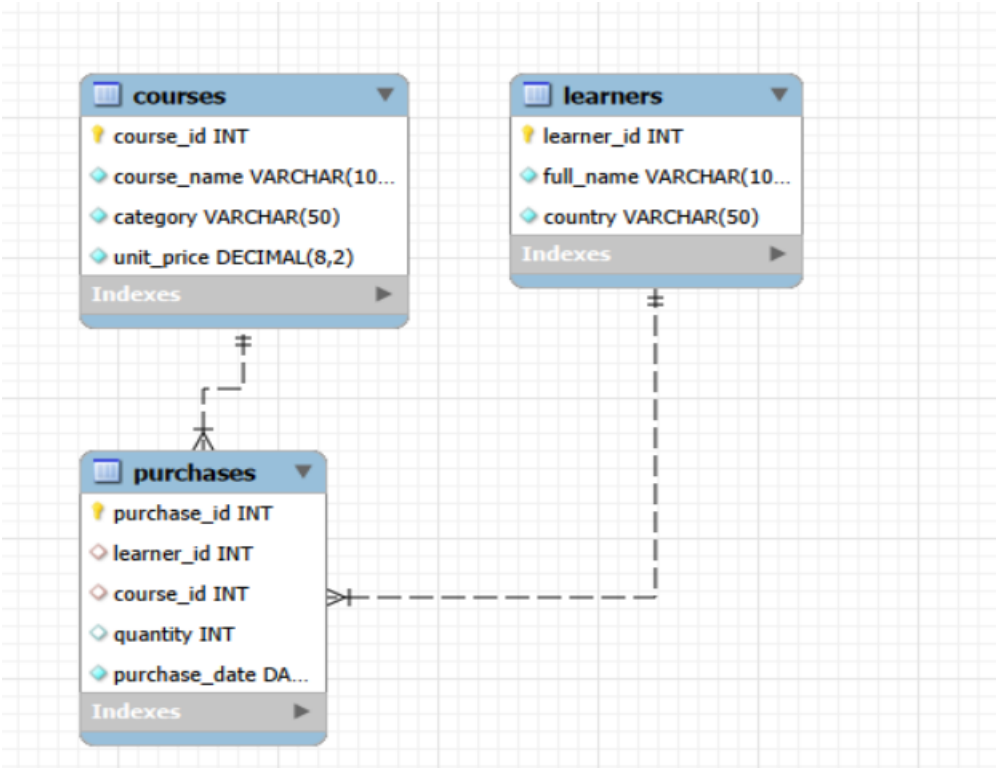
| category | total_revenue | number_of_purchases | avg_revenue_per_purchase |
|---|---|---|---|
| Programming | 39000.00 | 4 | 9750.00 |
| Data Science | 15000.00 | 1 | 15000.00 |
| Marketing | 4000.00 | 1 | 4000.00 |
| AI | 20000.00 | 2 | 10000.00 |

tegory_performance_view 15

---

## ENHANCED ENTITY–RELATIONSHIP DIAGRAM:

This image shows the relationships between the tables in the database schema.



## KEY INSIGHTS (SMART + ANALYTICS APPROACH):

## DESCRIPTIVE ANALYSIS:

- All five courses were purchased at least once, ensuring full dataset coverage.
- Programming courses **(*Python Basics, Web Development*)** recorded the highest purchase counts.
- **AI (*Machine Learning*)** and **Data Science** courses contributed the highest revenue due to premium pricing.
- **Marketing (*Digital Marketing*)** showed the lowest demand compared to technical categories.
- Revenue distribution highlights Programming as volume-driven and **AI/Data Science** as value-driven.
- Overall, technical courses dominate both learner interest and profitability.

## DIAGNOSTIC ANALYSIS:

- *Python Basics* (Programming) recorded strong demand with **₹150,000 revenue** from multiple purchases.
- *Web Development* (Programming) contributed **₹240,000 revenue**, making Programming the highest-volume category.
- *Machine Learning* (AI) generated **₹200,000 revenue** from fewer transactions, showing high value due to premium pricing.
- *Data Science Fundamentals* (Data Science) achieved **₹150,000 revenue**, reflecting steady demand for advanced skills.
- *Digital Marketing* (Marketing) produced only **₹40,000 revenue**, confirming it as the weakest performer.
- Overall, **Programming drives purchase volume**, while **AI and Data Science drive profitability**, and **Marketing requires promotional focus**.

## PREDICTIVE ANALYSIS:

- Programming courses are expected to continue leading in **purchase volume**, with projected revenues rising to **₹450,000–₹500,000** as demand for entry-level skills grows.
- AI (*Machine Learning*) is likely to remain the **highest revenue driver**, potentially crossing **₹300,000+** due to premium pricing and industry demand.
- Data Science courses are projected to sustain steady growth, with revenues increasing to around **₹200,000–₹250,000** as learners seek advanced analytics skills.
- Marketing (*Digital Marketing*) may remain underperforming unless supported by targeted promotions, with revenues stagnating near **₹50,000–₹80,000**.
- Overall, **technical categories (Programming, AI, Data Science)** will dominate both learner interest and profitability, while **Marketing requires strategic intervention** to improve uptake.

## PRESCRIPTIVE ANALYSIS:

- **Promote Programming courses** (*Python Basics, Web Development*) through bundled offers or certifications to leverage their high learner demand.
- **Upsell premium courses** (*Machine Learning, Data Science Fundamentals*) by offering advanced learning paths, ensuring sustained revenue growth.
- **Revitalize Marketing courses** (*Digital Marketing*) with targeted campaigns, discounts, or updated content to improve learner uptake.
- **Introduce cross-category packages** (e.g., Programming + AI) to encourage multi-category purchases and maximize learner value.
- **Monitor seasonal trends** and align course launches or promotions with peak months to capture higher order volumes.
- **Focus on technical categories** as core revenue drivers, while strategically repositioning Marketing to balance portfolio performance.

---

## CONCLUSION:

Programming courses drive the highest learner demand and purchase volume, while AI and Data Science courses generate the most revenue due to premium pricing. Marketing courses remain the weakest performer, requiring promotional focus. Overall, technical categories dominate both engagement and profitability, positioning them as the platform's key growth drivers.

---